

问题

组件是什么？解析后是什么？对象？属性？列表？

Excel 解析

Excel 文件的基本组件

以文件 2026 年销售数据.xlsx 为例，Excel 从大到小的层级结构如下：

1. 工作簿 (Workbook)

- 定义：整个 Excel 文件本身,wb
- 实例：2026 年销售数据.xlsx
- 作用：承载所有工作表、数据、格式、公式，是 Excel 的顶级容器

2. 工作表 (Worksheet)

- 定义：工作簿底部的标签页,ws
- 实例：1 月销售、2 月销售、总汇总、默认 Sheet1
- 作用：对数据分类存储，一个工作簿可包含多个工作表

3. 行 (Row)

- 定义：横向排列的数据行，用数字编号,row
- 实例：第 1 行为表头（姓名、产品、销售额），第 2~100 行为业务数据
- 作用：表示单条完整记录

4. 列 (Column)

- 定义：纵向排列的数据列，用英文字母编号,col
- 实例：A 列 = 姓名、B 列 = 产品名称、C 列 = 销售金额、D 列 = 销售日期
- 作用：表示同一类属性字段

5. 单元格 (Cell)

- 定义：行与列交叉的最小数据单元 cell= row[a]col[b]
- 实例：A1= 销售人员、B5= 笔记本电脑、C8=9600、D3=2026/3/1
- 包含内容：文本 / 数值 / 日期 / 公式 (=SUM(C2:C10)) / 批注 / 格式

6. 单元格区域 (Range)

- 定义：连续的多个单元格组合（合并单元格）
- 实例：A1:D100（整张销售数据表）、C2:C10（销售额列）

Openpyxl 解析（面向对象）

Openpyxl 是真正的 Excel 编辑器，所有东西都是类（对象），负责文件结构解析

- 专门读写新版 Excel（.xlsx）
- 可精确操作单元格、行、列、格式、公式
- 适合需要逐行逐单元格处理的场景
- 适用场景：需要修改单元格格式、插入公式、逐单元格控制的场景。

1. 工作簿（Workbook）

Python 类型： openpyxl.workbook.workbook.Workbook **类（对象）**

获取方式： wb = openpyxl.load_workbook("file.xlsx",
read_only=True, data_only=True)

"file.xlsx"作用：要打开的 Excel 文件路径 / 文件名（.xlsx / .xlsm）

read_only=True（只读模式）：以只读、高性能模式打开 Excel，只允许读取，不允许修改 / 保存，打开大文件（几万 / 几十万行）速度极快，占用内存极小

data_only=True（只读取数据）：控制“单元格里到底读什么内容”，Excel 单元格有两种内容：公式：=SUM(A1:A10)和公式计算后的值：1000

data_only=True： 只读计算后的结果（你看到的数字 / 文字）

data_only=False（默认）： 只读公式本身（看不到计算结果）

- 常用属性
 - wb.sheetnames：列表 → 所有工作表名
 - wb.active：对象 → 当前活动表
 - wb.worksheets：列表 → 所有工作表对象
 - wb.read_only：是否只读
- 常用方法
 - wb["Sheet1"] → 获取工作表
 - wb.create_sheet() → 创建表

- `wb.save()` → 保存
- `wb.close()` → 关闭

`workbook` 是一个「对象」，它属于 `openpyxl` 里的 `Workbook` 类但类本身不能用 `[]`，实例化后的对象可以用 `[]`

```
sheet = workbook[sheet_name]
```

等价于：

```
sheet = workbook.__getitem__(sheet_name)
```

这就是 Python 的 `[]` 运算符重载（索引语法）

只要一个类实现了：

```
def __getitem__(self, key):
```

它的对象就能用 `对象[key]` 语法！

2. 工作表（Worksheet）

Python 类型： `openpyxl.worksheet.worksheet.Worksheet` 类（对象）

获取方式：`ws = wb["Sheet1"]`

• 常用属性

- `ws.title`：表名
- `ws.max_row`：最大行数
- `ws.max_column`：最大列数
- `ws.merged_cells`：合并单元格管理器

• 常用方法

- `ws.cell(row=1, column=1)` → 获取单元格
- `ws.iter_rows()` → 按行读取
- `ws.merge_cells()` → 合并单元格

3. 行（Row）

Python 类型： `tuple`（元组）

获取方式：

```
for row in ws.iter_rows(min_row=2, values_only=True): # 从第 2 行开始读
    name, product, price, date = row
    print(name, product, price)
    读取前 N 行
```

```
iter_rows(max_row=5)
```

结构 (值 1, 值 2, 值 3, ...)

4. 列 (Column)

Python 类型: tuple (元组)

获取方式

```
for col in ws.iter_cols(values_only=True):
```

5. 单元格 (Cell)

Python 类型: openpyxl.cell.cell.Cell 类 (对象)

获取方式

```
cell = ws['A1']  
cell = ws.cell(row=1, column=1)
```

常用属性

- cell.value: 单元格值
- cell.row: 行号
- cell.column: 列号
- cell.coordinate: 坐标 (A1)

4. 处理空值

- openpyxl: if cell is None

6. 合并单元格 (MergedCell)

Python 类型: openpyxl.worksheet.cell_range.MultiCellRange 类 (对象)

获取方式

```
merged_ranges = list(ws.merged_cells.ranges)
```

属性

- min_row、max_row
- min_col、max_col
- coord → 范围如 A1:C1

#6.取消合并单元格

基本代码

```

from openpyxl import load_workbook, Workbook
from openpyxl.styles import Font, PatternFill

# =====
# 1. 用 openpyxl 创建一个带合并单元格、公式的 Excel 文件
# 演示：工作簿(Workbook) → 工作表(Worksheet) → 单元格(Cell)
# =====

wb = Workbook()          # 创建工作簿对象
ws = wb.active           # 获取活动工作表
ws.title = "销售明细"    # 修改工作表名

# 写入表头
headers = ["销售人员", "产品", "销售额", "奖金"]
ws.append(headers)        # 追加整行

# 写入示例数据
data = [
    ("张三", "A 产品", 12500, "=C2*0.1"), # 奖金 = 销售额*0.1 (公式)
    ("李四", "B 产品", 15800, "=C3*0.1"),
    ("王五", "A 产品", 13200, "=C4*0.1"),
    ("赵六", "C 产品", 17900, "=C5*0.1"),
]
for row_data in data:
    ws.append(row_data)

# 合并标题行上方的单元格（演示合并单元格）
ws.merge_cells("A1:D1")    # 合并 A1:D1
ws["A1"] = "2026 年第一季度销售报表" # 合并后左上角赋值
ws["A1"].font = Font(bold=True, size=14)

# 保存初始文件（先保存，后面再用 pandas 和 openpyxl 分别读取）
wb.save("销售数据.xlsx")
wb.close()
print("✔ 已创建文件：销售数据.xlsx（包含合并单元格、公式）\n")

# =====
# 2. openpyxl 面向对象方式逐单元格处理
# 特点：可以读写公式、修改格式、处理合并单元格等底层细节。
# =====

print("\n【Openpyxl 面向对象操作部分】")
# 2.1 以只读模式打开大文件（演示 read_only、data_only 参数）
wb_readonly = load_workbook(
    "销售数据.xlsx",
    read_only=True,      # 只读模式，无法保存，但速度快、内存小

```

```

        data_only=False          # 不取公式的值，取公式本身（为了演示公式读取）
    )
    ws_readonly = wb_readonly["销售明细"]

    print(f"工作表名: {ws_readonly.title}")
    print(f"最大行: {ws_readonly.max_row}, 最大列: {ws_readonly.max_column}")

    # 演示读取公式（因为 data_only=False）
    cell_formula = ws_readonly["D3"].value    # D3 是李四的奖金单元格
    print(f"D3 单元格原始内容（公式）: {cell_formula}")

    # 关闭只读工作簿（必须 close 释放资源）
    wb_readonly.close()
    print("只读工作簿已关闭。")

#2.2 用读写模式打开，修改单元格样式、取消合并单元格
wb_rw = load_workbook("销售数据.xlsx")
ws_rw = wb_rw["销售明细"]

# 处理合并单元格 - 获取合并区域并取消
print(f"\n 合并单元格区域: {list(ws_rw.merged_cells.ranges)}") # merged_cells.ranges
ws_rw.unmerge_cells("A1:D1")          # 取消合并

# 遍历行，为“高销售额”标红加粗
for row in ws_rw.iter_rows(min_row=2, max_col=4, values_only=False):
    sales_cell = row[2]    # 第3列（销售额）
    if sales_cell.value and sales_cell.value > 15000:
        name_cell = row[0]
        name_cell.font = Font(color="FF0000", bold=True)    # 红色加粗

# 保存新文件
new_file = "销售数据_修改后.xlsx"
wb_rw.save(new_file)
wb_rw.close()
print(f"✓ 已生成带格式和评级的文件: {new_file}")

```

pandas 解析（数据分析）

Pandas 不操作 Excel 本身，而是把 Excel 读成一张内存数据表,负责数据展示与处理

- 快速读取整张 / 多张工作表
- 直接转为表格结构（DataFrame），支持筛选、统计、聚合

- 支持读写 .xlsx / .xls, 语法简洁
- 适用场景: 批量数据分析、报表统计、数据清洗, 90% 数据场景首选。

1. 整个表格 (对应: 工作簿 + 1 个工作表)

Python 类型: pandas.core.frame.DataFrame 类 (对象)

获取方式 (默认读取第一个工作表)

```
df = pd.read_excel("file.xlsx")

# 读取指定工作表
df = pd.read_excel("2026 年销售数据.xlsx", sheet_name="1 月销售")
```

2. 列 (Column)

Python 类型: pandas.core.indexes.base.Index 对象

转列表

```
df.columns.tolist() # → 普通列表
df.shape[1]
```

3. 行 (Row)

Python 类型: pandas.core.series.Series 对象

```
for index, row in df.iterrows():
    df.shape[0]
    读取前 N 行

df.iloc[:5]
```

4. 单元格 (Cell)

Python 类型: Python 原生类型 (int/str/float)

获取方式

```
df.iloc[行, 列]
处理空值
```

```
pd.isna(cell) / df.fillna("")
```

5. 合并单元格

Pandas 不识别合并单元格！ 读进去后：

- 左上角有值
- 其他都是 NaN

6. 数据解析：

筛选销售额>5000 的数据

```
result = df[df["销售额"] > 5000]
```

统计每个销售的总销售额

```
group_result = df.groupby("销售人员")["销售额"].sum()
```

```
preview_data = df.iloc[:min(n_rows, df.shape[0])].fillna("")
```

· **截取数据：**取数据框 df 的前 N 行（N 是你设定的 n_rows，如果数据总行数不足 N，就取全部数据）；

· **空值替换：**把截取后数据中的所有缺失值（NaN/None）替换成空字符串 ""；最终得到一个干净、无缺失值的预览数据 preview_data。

基本代码

```
import pandas as pd
```

```
# ===== 1. 写入示例数据（pandas 负责生成） =====
```

```
df_original = pd.DataFrame({
```

```
    "销售人员": ["张三", "李四", "王五", "赵六", "孙七"],
```

```
    "部门": ["销售部", "技术部", "销售部", "技术部", "销售部"],
```

```
    "销售额": [12500, 15800, 13200, 17900, 14600],
```

```
    "奖金": [1250, 1580, 1320, 1790, 1460]
```

```
})
```

```
df_original.to_excel("绩效数据.xlsx", sheet_name="一月绩效", index=False)
```

```
print("✔ 用 pandas 生成示例 Excel 文件\n")
```

```
# ===== 2. Pandas 数据分析 -> 展示常用方法 =====
```

```
print("【Pandas 核心方法演示】")
```

```
# 读取指定工作表
```

```
df = pd.read_excel("绩效数据.xlsx", sheet_name="一月绩效")
```

```
# 基础信息
```

```
print("\n1. df.head(2) : 前 2 行数据")
```

```
print(df.head(2))
```

```
print("\n2. df.shape : 数据形状 (行数, 列数)")
```

```
print(df.shape)
```

```
print("\n3. df.columns.tolist() : 列名列表")
print(df.columns.tolist())

print("\n4. df.info() : 数据类型与非空统计")
df.info()

print("\n5. df.describe() : 数值列统计摘要")
print(df.describe())

# 数据选择与切片
print("\n6. df['销售额'] : 选择单列")
print(df["销售额"].head(2))

print("\n7. df[['销售人员','销售额']] : 选择多列")
print(df[["销售人员", "销售额"]].head(2))

print("\n8. df.iloc[:3, :2] : 按位置切片 (前 3 行, 前 2 列)")
print(df.iloc[:3, :2])

# 条件筛选
print("\n9. df[df['销售额'] > 15000] : 筛选销售额 > 15000")
print(df[df["销售额"] > 15000])

# 缺失值处理 (模拟一个空值)
df.loc[2, "奖金"] = None
print("\n10. 原始有缺失:\n", df[["销售人员", "奖金"]].head(3))
df["奖金"] = df["奖金"].fillna(0) # fillna() 填充缺失
print("    fillna(0) 后:\n", df[["销售人员", "奖金"]].head(3))

# 分组聚合
print("\n11. groupby('部门')['销售额'].sum() -> 部门销售总额")
grouped = df.groupby("部门")["销售额"].sum()
print(grouped)

# 多列聚合
agg_result = df.groupby("部门").agg(
    总销售额=("销售额", "sum"),
    平均奖金=("奖金", "mean")
)
print("\n12. agg() 多列聚合:\n", agg_result)

# 排序
df_sorted = df.sort_values("销售额", ascending=False)
```

```
print("\n13. sort_values 降序（销售额）:\n", df_sorted[["销售人员","销售额"]])
```

输出统计后的结果至新 Excel

```
grouped.to_excel("部门统计.xlsx", header=True)
```

```
print("\n14. 已将聚合结果写入 '部门统计.xlsx'")
```

总结表

Excel 概念	OpenPyXL 类型	Pandas 类型
工作簿	Workbook 类	无（直接读工作表）
工作表	Worksheet 类	DataFrame 类
行	tuple	Series 类
列	tuple	Index 类
单元格	Cell 类	普通值（int/str）
合并单元格	MultiCellRange 类	不支持，读成 NaN

xlrd（读取旧版 .xls/ 老格式文件）

核心功能

- 专门读取老版 Excel（.xls）
- 只读，不可写，轻量高效

```
import xlrd
```

```
# 1. 打开工作簿
```

```
wb = xlrd.open_workbook("旧版员工表.xls")
```

```
# 2. 获取工作表
```

```
ws = wb.sheet_by_name("Sheet1")
```

```
# 3. 获取行数、列数
```

```
rows = ws.nrows
```

```
cols = ws.ncols
```

```
# 4. 循环读取每行数据 for i in range(1, rows):
```

```
    row_data = ws.row_values(i)
```

```
    print(row_data)
```

csv 模块解析

核心功能

- Excel 可另存为 .csv 纯文本格式
- 用 Python 内置 csv 模块读取，无第三方依赖
- 速度极快，适合超大表格

```
import csv
with open("销售数据.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    # 跳过表头
    next(reader)
    for row in reader:
        print(row)
```

方法对比总结

解析方式	核心优势	适用场景
Excel 手动筛选	无需代码，上手快	少量数据、临时查看
数据透视表	一键汇总统计	快速做报表、多维度分析
pandas	语法简洁，数据分析能力极强	批量处理、自动化统计
openpyxl	单元格级精细控制，支持写回	修改 Excel 格式、插入公式
xlrd	兼容老 .xls，轻量	读取旧版 Excel 文件
csv 模块	无依赖、速度快	超大文件、跨平台数据交换

Excel 解析核心难点痛点、优质学习资源与开源项目全指南

Excel 解析的核心难点与痛点

Excel 本质是“半结构化 + 富格式”文档，而非标准数据表 —— 所有解析难题均源于此，方案需根据**表格复杂度、数据量、是否需要写回格式**三要素权衡选择。

类别	合并的核心问题	典型表现与后果
表格结构不规则 (最大痛点)	多级表头、非二维报表、分块数据、动态行列	标准解析工具无法直接适配，需大量规则处理行列偏移；数据区域不连续，易混入无效数据或丢失关键信息
合并单元格陷阱	值丢失、复杂嵌套合并、写回还原困难	合并区域内仅左上角有值，其余为 NaN；修改后几乎无法完美还原原格式
数据类型与格式污染	公式与值矛盾、类型漂移（长数字/日期）、富文本/特殊	公式结果未必可读；身份证号丢失精度；换行符、不可见字符破坏后续处

类别	合并的核心问题	典型表现与后果
	字符	理
大文件性能瓶颈	内存爆炸、流式读取局限、写入慢且易损坏	默认全量加载导致 OOM；只读模式无法处理复杂结构；百万行级文件处理困难
兼容性与企业级工程难题	格式不统一（xls/xlsx/xlsm）、软件差异、加密/损坏文件；批量容错、双向同步、数据校验	不同引擎解析结果不一致；批量处理缺乏稳定性；修改后保留原格式几乎不可能

优质学习资源与文章推荐

2. 国内深度技术文章

《Rag chunk 之：Excel 文档解析》（稀土掘金）地址：

<https://juejin.cn/post/7512361858344550412> 核心内容：合并单元格的标准化处理方案、公式与空值清洗、非结构化 Excel 的 RAG 适配，有完整可运行的代码实现

《千万行 Excel 文件解析的性能优化实战》（阿里云开发者社区）核心内容：Java/Python 大文件 Excel 解析的内存优化方案、流式读取的坑与解决方案、生产环境调优参数

《非结构化 Excel 报表的自动化解析方案》（腾讯云开发者社区）核心内容：财务报表、业务审批表等非标表格的解析思路，规则引擎 + AI 辅助的落地实践

《Excel 解析的底层原理与工业级解决方案》（InfoQ）核心内容：xlsx 文件的底层压缩包结构、OOXML 格式解析、工业级系统的容错设计

3. 国外优质文章与经典书籍

《The Ultimate Guide to Excel File Handling in Python》（Towards Data Science）核心内容：Python 生态 Excel 工具全对比，不同场景的工具选型指南，从入门到工业级的完整教程

《Handling Large Excel Files in Python Without Crashing Your System》（Medium）核心内容：大文件 Excel 处理的流式方案、内存优化技巧，对比了 polars/calamine/openpyxl 的性能差异

经典书籍

- 《Python for Data Analysis》（pandas 作者 Wes McKinney 著）：数据分析领域圣经，包含 Excel 数据读取、清洗、结构化处理的权威方法论

- 《Automate the Boring Stuff with Python》：Python 办公自动化入门经典，Excel 处理章节有大量可直接复用的脚本
- 《OpenPyXL Guide》：openpyxl 官方电子书，深入讲解格式处理、合并单元格、公式计算等进阶用法

2026 年活跃维护的优质开源项目

按技术栈和适用场景分类，优先推荐社区活跃、生产级可用的项目，避免推荐已停止维护的工具。

1. 基础通用解析库

项目	地址	核心特点	适用场景
openpyxl	https://foss.heptapod.net/openpyxl/openpyxl	Python 生态最主流的 xlsx/xlsm 处理库，纯 Python 通用 xlsx 文件读实现，支持读写、格式控制、写、格式处理、合并单元格、公式处理，社区并单元格操作持续活跃	
pandas	https://github.com/pandas-dev/pandas	数据分析标配，基于 openpyxl/xlrd 等引擎，一行代码读取标准二维表，支持分块读取、数据清洗、格式转换	标准结构化 Excel 的数据分析、预览、快速处理，适合你做数据预览、转 Markdown 的场景
XlsxWriter	https://github.com/jmcnamara/XlsxWriter	专注于 xlsx 文件写入，支持丰富的单元格格式、图表、公式、合并单元格，写入性能远超 openpyxl	生成格式复杂的 Excel 报表、批量写入数据
pyxlsb	https://github.com/willtrnr/pyxlsb	唯一支持 xlsb 二进制 Excel 格式的 Python 库，大文件体积比 xlsx 小 50%，读取性能更好	处理 xlsb 格式的大文件
xlrd	https://github.com/python-excel/xlrd	老一代 Excel 库，2.0+ 版本仅支持 xls 二进制文件读取，不再支持 xlsx	处理老旧的 xls 格式文件

2. 大文件高性能解析库

项目	地址	核心特点
calamine-python	https://github.com/tafia/calamine	Rust 编写的 Python 绑定，Excel 解析速度比 openpyxl 快 10 倍以上，内存占用仅为几分之一，支持 xls/xlsx/xlsm/xlsb 全格式，流式读取百

项目	地址	核心特点
		万行文件无压力
polars	https://github.com/pola-rs/polars	Rust 编写的高性能数据分析库，比 pandas 快几十倍，内存优化极致，原生支持 Excel 读取，是大文件数据分析替代 pandas 的首选
PyExcelerate	https://github.com/kz26/PyExcelerate	Python 写入 Excel 最快的库之一，比 openpyxl 快 5-10 倍，支持大文件批量写入、格式控制

3. 非结构化 / 非标 Excel 解析库

项目	地址	核心特点
unstructured	https://github.com/Unstructured-io/unstructured	AI 驱动的非结构化文档解析标杆，支持 Excel/PDF/Word 等格式，自动识别表格、标题、备注、合并单元格，无需手动写规则即可提取非标报表中的结构化数据，完美适配 RAG 场景
excel2json	https://github.com/sicara/excel2json	轻量级工具，自动处理多层表头、合并单元格填充，把非标 Excel 转换成结构化 JSON，配置简单易上手
camelot-py	https://github.com/camelot-dev/camelot	原本专注 PDF 表格提取，现已支持 Excel 不规则表格识别，能自动定位数据区域，处理多表格穿插的复杂 Sheet

4. 公式计算与高级处理库

项目	地址	核心特点
formualizer	https://github.com/PSU3D0/formualizer	Rust 核心的生产级 Excel 公式引擎，支持 320+Excel 原生函数，支持 Python/JS 多语言绑定，增量计算、动态数组支持，解决 openpyxl 无法计算公式的痛点
xlwings	https://github.com/xlwings/xlwings	直接调用本地 Excel/WPS 引擎，公式计算 100% 兼容 Excel 原生功能，支持读写、宏调用、格式保留，适合需要和 Excel 深度交互的场景，缺点是需要安装 Excel/WPS
pycel	https://github.com/dgorissen/pycel	把 Excel 公式编译成 Python 代码，支持离线计算复杂 Excel 模型，无需依赖 Excel 引擎

5. 批量处理工具集

- pyexcel: <https://github.com/pyexcel/pyexcel> 统一的 Excel 处理接口，封装了不同底层引擎，一行代码兼容 xls/xlsx/xlsm/ods 等所有格式，批量处理文件极其方便
- openpyxl-utils: <https://github.com/buke/openpyxl-utils> openpyxl 的扩展工具集，简化合并单元格处理、行列遍历、格式批量修改，减少重复代码

4. AI 驱动新兴项目

- **Excel MCP Server** : <https://github.com/ModelContextProtocol/server-excel> 基于 MCP 协议的 AI 代理工具，让大模型通过自然语言直接操作 Excel，无需写代码即可完成数据提取、分析、格式修改，适合 AI Agent 场景
- **Univer**: <https://github.com/dream-num/univer> 前飞书表格团队开源的在线表格引擎，支持 Excel 全格式解析、AI 自然语言操作，兼容 Excel 公式、格式、合并单元格，适合构建自定义表格系统

不如语冰